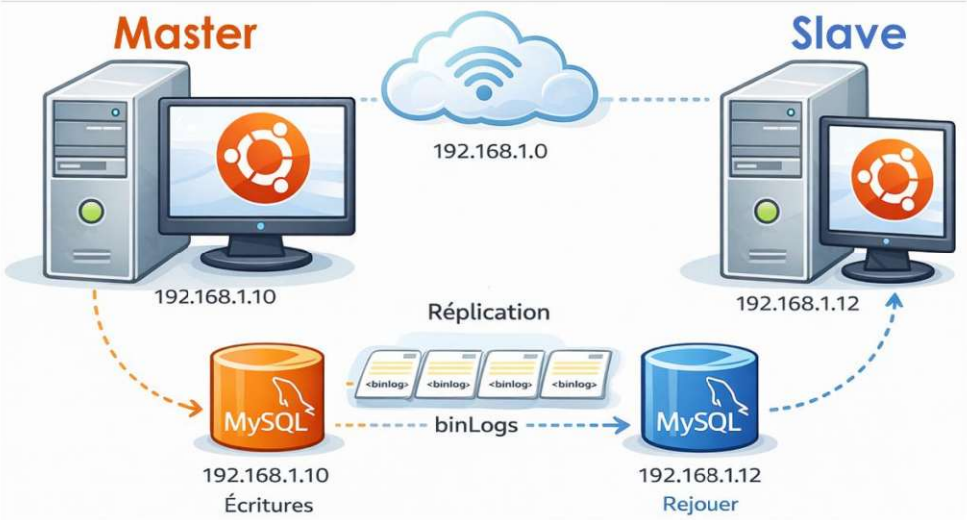
DUT - Ingénierie Logicielle 2025/2026

Administration des infrastructures et des services

Travaux Pratiques

TP3 - Mise en place d'une réplication MySQL Master → Slave

Ce TP vous apprend à mettre en place une **réplication unidirectionnelle** et **asynchrone** entre deux serveurs MySQL. Le serveur **Master** accepte les écritures et enregistre les modifications dans ses **binary logs** (binlogs). Le serveur **Slave** se connecte au Master, lit les binlogs, puis **rejoue** les événements pour obtenir une copie quasi-identique des données.



Vous devrez produire une preuve de synchronisation (INSERT/UPDATE/DELETE visibles sur le Slave) et analyser ce que la réplication apporte.

---

1) Environnement et prérequis

Machines

Deux machines Linux Ubuntu 22.04 sur un réseau qui permet la communication directe :

- **VM1 : Master.** @ `192.168.1.10`
- **VM2 : Slave**. @ `192.168.1.12`

---

2) Objectifs pédagogiques

À la fin du TP, vous devez savoir :

1. Expliquer la différence entre **réplication asynchrone** et **réplication synchrone**.

2. Expliquer le rôle du **binlog** et de la notion de **File/Position**.
3. Configurer le Master : `server-id`, `log_bin`, format de binlog, écoute réseau.
4. Créer un utilisateur dédié `repl` avec les privilèges de réplication.
5. Configurer le Slave : `server-id`, paramètres read-only, `CHANGE MASTER TO`.
6. Démarrer la réplication, vérifier son état, diagnostiquer les erreurs.
7. Tester la synchronisation sur des opérations réelles.
8. Discuter le failover manuel (sans le mettre en production).

---

3) Contexte théorique

La réplication MySQL Master–Slave classique est **asynchrone** : le Master confirme une écriture sans attendre que le Slave l'ait appliquée. On gagne en simplicité et performance, mais on accepte un risque de **lag** (retard) et de perte potentielle de transactions si le Master tombe avant que le Slave ait rattrapé.

La réplication **n'est pas une sauvegarde** : toute suppression, corruption logique ou attaque (ex. ransomware) est répliquée au Slave.

PARTIE A — Vérifications réseau

Exercice A1 — Vérifier connectivité et services

1. Depuis le Slave, vérifier qu'on atteint le Master (ping et test du port MySQL 3306)
2. Vérifier que MySQL tourne sur chaque nœud.

Livrable A1 : capture ou sortie montrant que le Slave peut joindre le Master sur 3306.

PARTIE B — Configuration du Master

Exercice B1 — Activer binlog et config réseau

Sur le Master, vous devez :

1. Fixer un identifiant serveur unique : `server-id=1`
2. Activer les binlogs : `log_bin=...`
3. Forcer un format robuste : `binlog_format=ROW`
4. Faire écouter MySQL sur le réseau (pas seulement `127.0.0.1`) → l'objectif est que le Slave puisse se connecter.

Livrable B1 : sorties prouvant :

- `server_id`
- `log_bin`
- `binlog_format`
- et l'écoute réseau sur 3306

Exercice B2 — Créer un utilisateur de réplication

Créer un compte `repl` autorisé depuis l'IP du Slave et lui attribuer les privilèges de réplication.

Attention MySQL 8 : selon la config, le plugin d'authentification par défaut peut provoquer une erreur "secure connection required". Votre solution doit être fonctionnelle.

Livrable B2 : preuve que le compte existe (`user`, `host`) et que la réplication pourra s'authentifier.

PARTIE C — Initialisation cohérente du Slave

Exercice C1 — Créer une base de test sur Master

Créer une base `tp_replication` avec une table `clients` :

- champs : `id auto-incrément`, `nom`, `ville`
- insérer au moins 2 lignes

Livrable C1 : capture du `SELECT * FROM clients` sur Master.

Exercice C2 — Faire un dump cohérent incluant la position binlog

Générer un dump cohérent de la base **en incluant** la position `MASTER_LOG_FILE` et `MASTER_LOG_POS`.

Livrable C2 :

- un extrait du dump contenant `MASTER_LOG_FILE` et `MASTER_LOG_POS`.

Exercice C3 — Importer sur le Slave

1. Copier le dump sur le Slave
2. Importer le dump sur le Slave
3. Vérifier que la base et les données existent bien sur le Slave **avant** de démarrer la réplication.

Livrable C3 : `SELECT * FROM clients` sur Slave montrant les données initiales.

PARTIE D — Configuration et démarrage de la réplication sur Slave

Exercice D1 — Configurer le Slave

Configurer un `server-id` différent de celui du Master (ex. `2`) et activer un mode read-only (pour éviter les écritures accidentelles sur le Slave).

Livrable D1 : preuve de `server-id` du Slave et justification du mode read-only.

Exercice D2 — Déclarer le Master (CHANGE MASTER TO)

Utiliser les valeurs `MASTER_LOG_FILE` et `MASTER_LOG_POS` obtenues dans le dump pour déclarer le Master au Slave.

Livrable D2 : capture ou sortie montrant la configuration (master host, user, log file/pos).

Exercice D3 — Démarrer et vérifier

1. Démarrer la réplication
2. Vérifier que les deux threads sont actifs :

- IO : connexion/lecture binlog
- SQL : application des événements

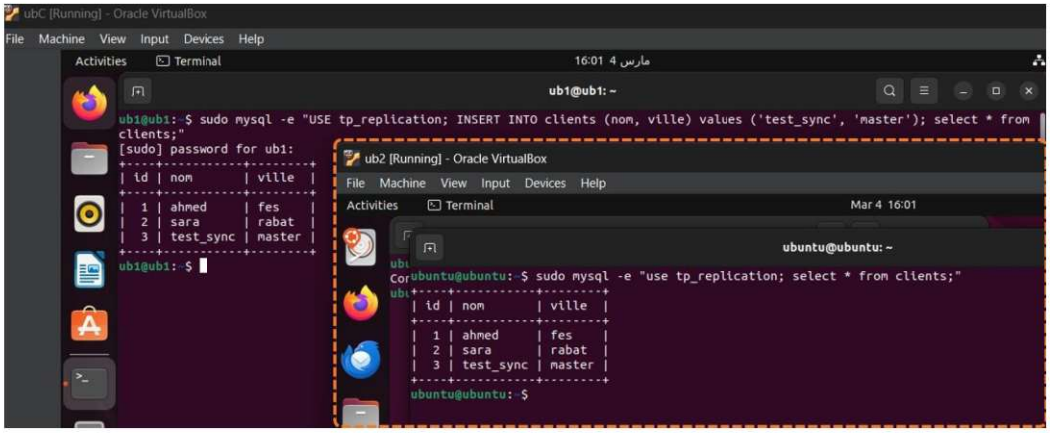
Livrable D3 : sortie de statut prouvant :

- **IO = Yes**
- **SQL = Yes**
- **absence d'erreurs**
- **`Seconds_Behind_Master` (ou équivalent)**

PARTIE E — Test de synchronisation et validation

Exercice E1 — Test d'écriture

1. **Sur Master : insérer une nouvelle ligne dans `clients`**
2. **Sur Slave : vérifier qu'elle apparaît**



Refaire au moins :

- 1 INSERT
- 1 UPDATE
- 1 DELETE

Livrable E1 : captures comparatives Master/Slave prouvant que les mêmes données sont présentes.

Exercice E2 — Mesurer et discuter le lag

1. Relever `Seconds_Behind_Master` (ou indicateur équivalent)
2. Expliquer ce qui peut augmenter ce retard (charge CPU, IO disque, réseau, grosses transactions…)

Livrable E2 : court paragraphe argumenté.

PARTIE F — Résilience et failover manuel (réflexion)

Sans mettre en place un cluster automatique, répondre :

1. Si le Master tombe, comment promouvoir le Slave en nœud actif (conceptuellement) ?
2. Quels risques si l'ancien Master revient et accepte des écritures en parallèle ?
3. Pourquoi une réplication Master–Slave seule ne garantit pas une HA automatique ?

---

4) Questions de réflexion (réponses attendues)

1. Pourquoi la réplication MySQL est souvent asynchrone ? Quels compromis ?
2. En multi-maîtres, quel est le risque majeur des écritures simultanées ?
3. Pourquoi la réplication ne remplace pas une sauvegarde ?
4. Quelles mesures minimales de sécurité appliquer au compte de réplication ?

---

5) Livrables finaux (à rendre)

1. Un compte rendu PDF contenant :

   - les commandes essentielles utilisées avec leurs interprétations
   - toutes les captures d'ecran
   - les preuves Master/Slave (INSERT, SELECT)
   - le statut de réplication (IO/SQL)
   - un paragraphe sur le lag
   - *Réponses aux questions de réflexion.*